

Experiment 1: Terrain Mapping

Objective: Generate aerial terrain information during glider descent using a downward-facing camera. Sequential images/frames are captured, geotagged with GPS + barometric altitude + IMU orientation, and processed onboard into a terrain map or image mosaic stored on the microSD card.

Method

The camera (mounted behind a transparent bottom window) captures images at a controlled frame rate. The Raspberry Pi performs timestamping, frame indexing, image compression, metadata tagging, image alignment, and terrain file generation. Status (image count, camera state, GPS, altitude, processing status) is relayed to the ground station via telemetry.

Expected Outputs

- Downward-facing terrain images/video
- Geotagged & altitude-tagged image dataset
- Onboard-generated terrain map / image mosaic
- Camera and gimbal stability analysis

5.4.4 Experiment 2: Guided Glider Descent

Objective: Demonstrate controlled descent via a servo-assisted autonomous glider. The drogue descent phase ends at ~600 m AGL when the glider is deployed; the payload then performs guided flight using IMU, GPS, and barometric feedback.

Method

BMP388 monitors altitude continuously. At ~600 m AGL, two MG996R servos push the folded glider out of its compartment. Once stable, the Raspberry Pi transitions to guided descent mode, commanding the MG996R wing servos based on attitude error and GPS heading. Servos default to neutral (90°); corrections applied only when drift is detected. Servo range limited to 60°–120° to prevent overstress.

Expected Outputs

- Glider deployment confirmation at ~600 m AGL
- Reduced landing dispersion vs. passive descent
- GPS descent trajectory & IMU stability data
- Deployment and steering servo command logs
- Landing dispersion and steering performance analysis

1. PAYLOAD

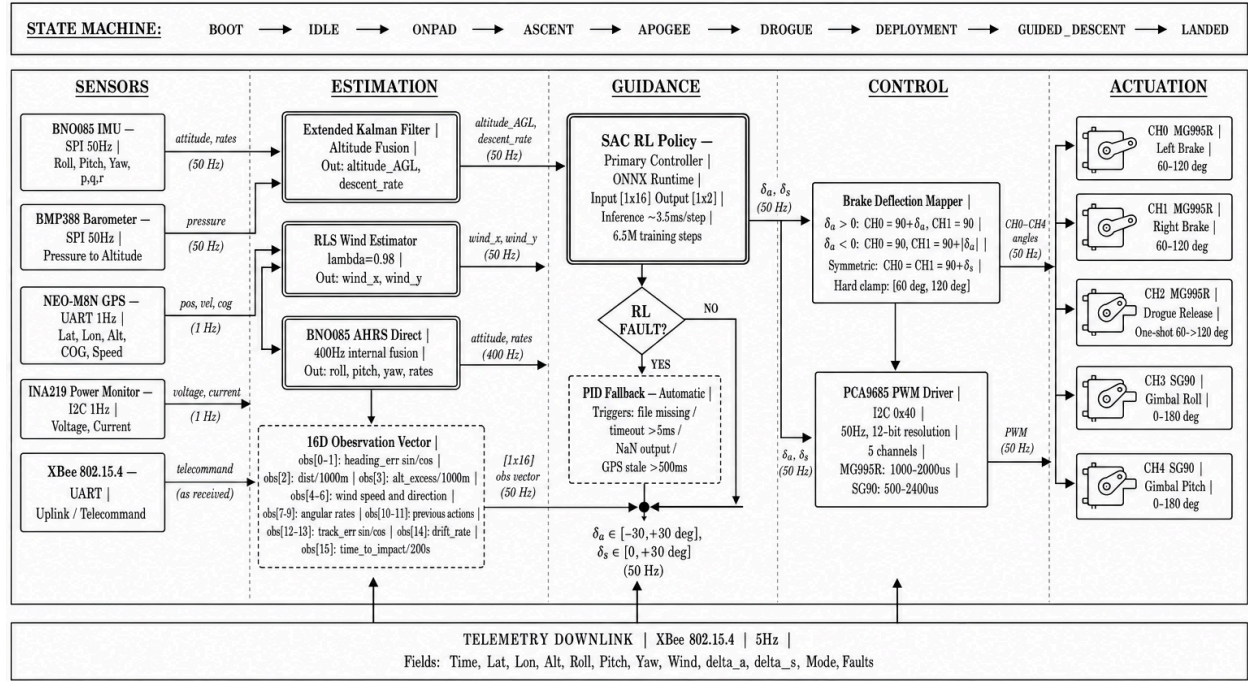


Fig. 1. GARUD GNC System Architecture. Data flow left-to-right. Raspberry Pi 4 host.

1.1 Payload Objective

The GARUD payload (CAN-7USAT) is a single-skin ram-air parafoil glider deployed from the rocket nosecone at apogee (~600 m AGL). Its primary mission objective is to autonomously navigate from the deployment point to a pre-programmed GPS landing target using onboard Guidance, Navigation, and Control (GNC) running on a Raspberry Pi 4 Model B (8 GB). A secondary concurrent experiment captures geotagged aerial imagery to produce a terrain map of the flight zone.

Primary success criterion: Landing within 20 m of the target GPS coordinate.

1.2 Payload Description

GARUD is a guided descent payload comprising the following hardware:

Component	Part	Interface	Role
Flight Computer	Raspberry Pi 5 Model B 8 GB	—	Runs all GNC, telemetry, terrain mapping
IMU	BNO085	SPI (CS=GPIO5)	Calibrated Euler angles, quaternions — built-in AHRS
Barometer	BMP388	SPI (CS=GPIO22)	Altitude measurement — AGL reference
GPS Receiver	u-blox NEO-M8N	UART (/dev/ttyAMA0)	Position, velocity, course-over-ground

Power Monitor	INA219	I2C (0x41)	Real-time bus voltage and current
Servo Driver	PCA9685 16-ch	I2C (0x40)	PWM generation for all 5 servos
Glider Servos	EMAX ES3004 $\times$ 3	PCA9685 CH0,1,2	Brake deflection (left, right) + drogue release
Gimbal Servos	SG90 $\times$ 2	PCA9685 CH3,4	Camera gimbal roll and pitch stabilisation
Telemetry	XBee 802.15.4	UART (GPIO14/15)	Real-time 2.4 GHz downlink to ground station
Camera	RPi HQ Camera (IMX477)	CSI	12.3 MP geotagged imagery for terrain mapping
Buzzer	BZ1 via 2N2219	GPIO16	Recovery audio beacon
HAT PCB	GARUD HAT Rev 2	40-pin header	Interfaces all sensors to RPi GPIO

Table 1. Payload Description

1.3 Payload Design

1.3.1 Canopy

Single-skin ram-air parafoil. Steered exclusively by asymmetric brake deflection — there are no ailerons, no elevators, and no rudder. The trailing edge of the canopy is mechanically linked to EMAX ES3004 servos via control lines. Servo rotation of 60° – 120° corresponds to 0–15 mm of brake line travel, sufficient to generate the full range of yaw rates required for navigation.

Turn model: $\psi = K_{\text{turn}} \times \delta a$

where δa is the asymmetric brake command (-30° to $+30^\circ$) and K_{turn} is the parafoil turn constant (~ 0.5 rad/s per degree of brake).

1.3.2 Software Stack

The flight software runs as a single Python process on the Raspberry Pi 4, with three concurrent threads: a GNC core thread at 50 Hz, a terrain mapping thread at 0.5 Hz, and a telemetry downlink thread at 5 Hz. All hardware access (I2C, SPI, UART) is centralised through a single SensorBus module to prevent bus conflicts between threads. The GNC thread executes the full pipeline — sensor reads, EKF fusion, RL inference, and servo commands — within each 20 ms cycle. The state machine runs in the main thread and controls which operations are active at each phase of flight.

All hardware access is centralized in SensorBus. No other module opens I2C, SPI, or UART directly. This prevents bus conflicts between the two concurrent experiments.

1.3.3 Control Loop Rates

Process	Rate
GNC control loop	50 Hz (20 ms)

BNO085 attitude polling	50 Hz
BMP388 EKF update	50 Hz
GPS position update	1 Hz
XBee telemetry downlink	5 Hz
Terrain imaging	0.5 Hz

Table 2. Control Loop Rates

1.4 Payload Operation

Pre-flight (on-pad):

1. Power on payload — flight computer boots, state machine enters BOOT.
2. BNO085 performs internal calibration (~30 s).
3. GPS acquires fix — operator confirms on ground dashboard.
4. Operator sets target_latitude and target_longitude in config/gains.yaml.
5. State transitions to ONPAD after operator arm command.

During ascent:

- Servos locked at neutral (90°).
- Barometer and IMU record baseline ground pressure.
- State machine monitors altitude for apogee detection.

At apogee (~600 m AGL):

- Deployment trigger fires (BMP388 10-sample moving average confirms altitude).
- Drogue release servo (CH2, EMAX ES3004) fires: 60° → 120°.
- Canopy deploys and inflates. State transitions to GUIDED_DESCENT.

During guided descent:

- SAC RL model runs at 50 Hz, commanding δa and δs via PCA9685.
- XBee downlinks telemetry at 5 Hz to ground station.
- Wind estimation updates every GPS fix (~1 Hz).

Landing:

- When altitude < 5 m AGL: symmetric flare command (both servos to 120°).
- Buzzer activates recovery beacon pattern.
- State transitions to LANDED.

1.5 Payload Integration

The GARUD HAT PCB mounts directly on the Raspberry Pi 5 GPIO header (40-pin). All sensors are soldered to or connected via JST connectors on the HAT. The camera is mounted on a two-axis SG90 gimbal below the chassis for downward-facing imagery. Canopy control lines are

routed through two fairleads and terminated at the EMAX ES3004 servo horns on CH0 (left) and CH1 (right). The drogue release line connects to CH2.

1.6 Payload Validation

Test	Method	Pass Criterion
BNO085 calibration	SPI scan + read Euler angles for 30 s	Roll/pitch/yaw within $\pm 2^\circ$ of reference
BMP388 altitude	Compare to known elevation (GPS)	AGL reading within ± 5 m at ground level
GPS fix acquisition	Cold start outdoors	3D fix within 90 s, HDOP < 2.0
Servo range	python tests/test_glider_servos.py $\rightarrow$ sweep test	All 5 servos respond; no binding or jitter
Deployment trigger	Simulate altitude in SITL mode	Drogue servo fires at 600 m AGL ± 5 m
SITL end-to-end	python flight_computer.py --sitr (500 drops)	$\geq 6\%$ success rate (< 20 m CEP)
XBee link budget	Bench test at 50 m range	All packets received; RSSI > -90 dBm
SAC RL inference on Pi	Run ONNX model, time 1000 steps	Mean inference < 5 ms; no NaN output

Table 3. Payload Validation

1.8 Payload Constraints

Constraint	Value	Source
Maximum deployment altitude	600 m AGL	Competition rules
XBee transmit window	90 min from launch detection	IN-SPACE frequency assignment
XBee main channel	Ch 11 (2405 MHz)	Team 002 assignment
XBee backup channel	Ch 21 (2455 MHz)	Team 002 assignment
Forbidden frequencies	433 MHz (ignition), 446 MHz (voice)	Competition rules
Target coordinate lock	Before ONPAD only — no mid-flight change	Software enforced
Max symmetric brake	$\delta s \leq 30^\circ$	Canopy structural limit
Max asymmetric brake	$\delta a \in [-30^\circ, +30^\circ]$	RL action space
Operating temperature	0°C to 45°C	RPi 4 thermal specification

Table 4. Payload Constraints

2. GNC SOFTWARE ARCHITECTURE

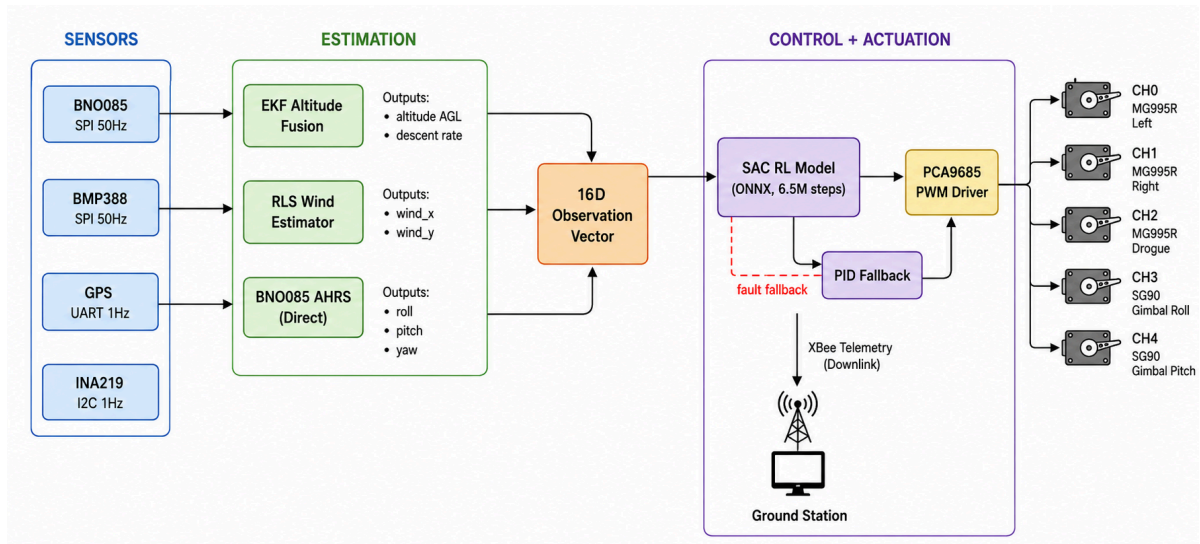


Fig 2. Software Architecture

3. MISSION STATE MACHINE

3.1 States

State	Entry Condition	Exit Condition	Servo Action
BOOT	Power-on	BNO085 calibrated + GPS fix	Locked neutral
IDLE	Calibration complete	Operator arm command	Locked neutral
ONPAD	Arm command received	Altitude > 10 m (liftoff)	Locked neutral
ASCENT	Liftoff detected	Altitude decreasing (apogee)	Locked neutral
APOGEE	Altitude decreasing	Deployment trigger fired	Locked neutral
DROGUE	Drogue servo fired	Canopy inflated (BMP stable)	CH2 = 120°
DEPLOYMENT	Canopy confirmed	Altitude < 580 m AGL	All neutral
GUIDED_DESCENT	Below 580 m AGL	Altitude < 5 m AGL	RL / PID active
LANDED	Altitude < 5 m AGL	Terminal (never)	Flare → buzzer on

Table 5. Mission States

3.2 State Transitions

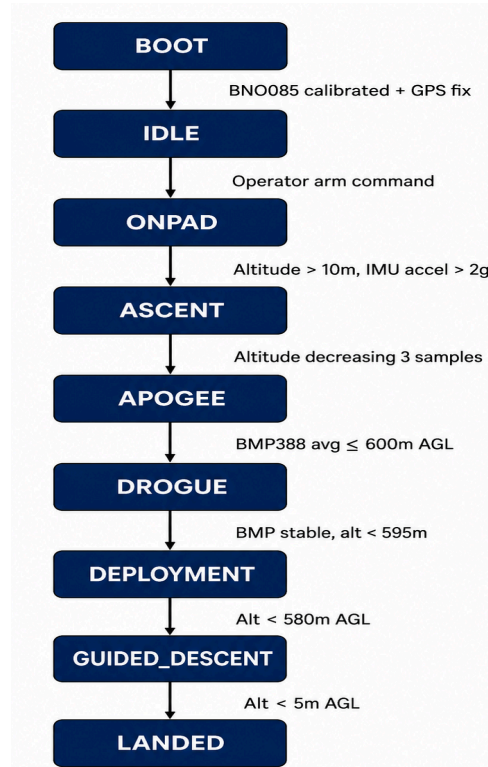


Fig 3. State Transition

3.3 Deployment Trigger

A 10-sample moving average of BMP388 barometric altitude (100 ms window) is used to trigger deployment. This filters barometric spikes at apogee and prevents false triggers during rapid pressure transients.

4. NAVIGATION ALGORITHM

4.1 Overview

The GNC system uses a Soft Actor-Critic (SAC) Reinforcement Learning model as the primary guidance law. A classical Direct-Heading PID is retained as an automatic silent fallback. The RL model was trained in a 4-DOF point-mass parafoil simulator for 6.5 million steps.

4.2 RL Observation Space (16-dimensional)

The observation vector is fixed — it must not be changed without retraining the model.

Index	Signal	Encoding	Normalization
0–1	Heading error (body yaw vs target)	sin / cos	$[-1, 1]$
2	Distance to target	scalar	$\div 1000 \text{ m}$
3	Altitude excess above target	scalar	$\div 1000 \text{ m}$

4	Wind speed	scalar	$\div 10$ m/s
5–6	Wind direction	sin / cos	$[-1, 1]$
7	Pitch rate	scalar	$\div 0.5$ rad
8	Roll rate	scalar	$\div 0.5$ rad
9	Yaw rate	scalar	$\div 0.5$ rad/s
10	Previous δa	scalar	$\div 30$
11	Previous δs	scalar	$\div 30$
12–13	Track error (GPS COG vs target heading)	sin / cos	$[-1, 1]$
14	Lateral drift rate	scalar	$\div 8$ m/s
15	Time to impact (capped at 200 s)	scalar	$\div 200$ s

Table 6. Reinforcement Learning Observation Space

4.3 Action Space

Output	Range	Physical meaning
delta_a	$[-30^\circ, +30^\circ]$	Asymmetric brake — left/right turn command
delta_s	$[0^\circ, +30^\circ]$	Symmetric brake — steepen glide / decelerate

Table 7. Action Space

ONNX raw output rescaling:

$$\text{delta_a} = \text{raw}[0] \times 30.0$$

$$\text{delta_s} = (\text{raw}[1] + 1.0) / 2.0 \times 30.0$$

4.4 Guidance Phases

Phase	Altitude	Strategy
Cruise	> 200 m AGL	Aggressive heading correction — maximise cross-range distance
Approach	50–200 m AGL	Balanced heading + altitude budget — begin final alignment
Final	< 50 m AGL	Heading-lock — minimal brake, preserve glide for last metres
Flare	< 5 m AGL	Both brakes to 120° — maximum deceleration for soft touchdown

Table 8. Guidance Phases

4.5 Deployed Model

Property	Value
File	models/sac_policy_6500000_16D.onnx
SHA256 (8 chars)	ff74f871
Input shape	[1, 16]
Output shape	[1, 2]
Mean inference	~3.5 ms on RPi 4 (< 5 ms limit)
Training steps	6,500,000

Table 9. Deployed Model

5. SENSOR FUSION

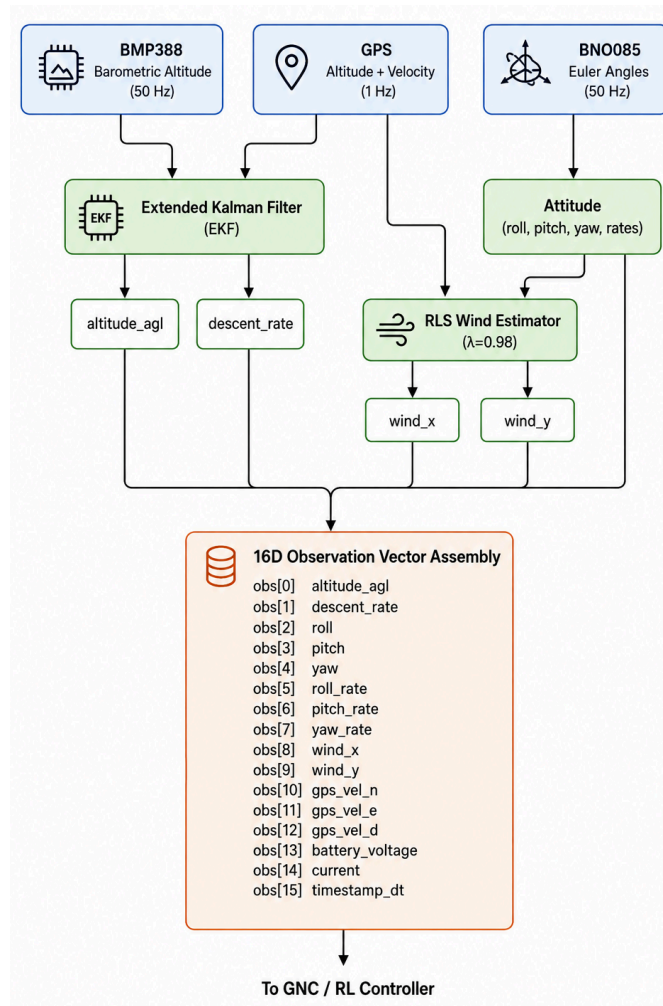


Fig 4. Sensor Fusion

5.1 Altitude — Extended Kalman Filter (EKF)

State vector: [altitude_agl, vertical_velocity]

Predict step (50 Hz):

$$\mathbf{x}_k|k-1 = [[1, dt], [0, 1]] \times [\text{alt}, v_z]$$

Update — BMP388 (50 Hz): $H = [1, 0]$, $R_{\text{baro}} = 0.5 \text{ m}^2$

Update — GPS altitude (1 Hz): $H = [1, 0]$, $R_{\text{gps}} = 9.0 \text{ m}^2$

Output: smooth 50 Hz altitude and descent rate. Used directly in obs[3] (altitude excess) and obs[15] (time to impact = altitude ÷ descent_rate).

5.2 Wind Estimation — Recursive Least Squares (RLS)

GPS ground velocity = airspeed vector + wind vector. At each GPS fix (1 Hz):

$$[v_{\text{gps_east}}] = [\text{airspeed} \times \cos(\text{heading})] + [\text{wind_x}]$$

$$[v_{\text{gps_north}}] \quad [\text{airspeed} \times \sin(\text{heading})] \quad [\text{wind_y}]$$

RLS updates the wind estimate [wind_x, wind_y] with forgetting factor $\lambda = 0.98$, tracking slowly changing wind throughout the descent. Output populates obs[4–6].

5.3 Attitude — BNO085 Direct Output

BNO085 has a built-in AHRS running sensor fusion at 400 Hz internally. The Pi reads pre-fused Euler angles (roll, pitch, yaw) via SPI at 50 Hz. No Madgwick or Mahony filter is applied — BNO085 already outputs fully fused, calibrated attitude.

6. SERVO CONTROL LOGIC

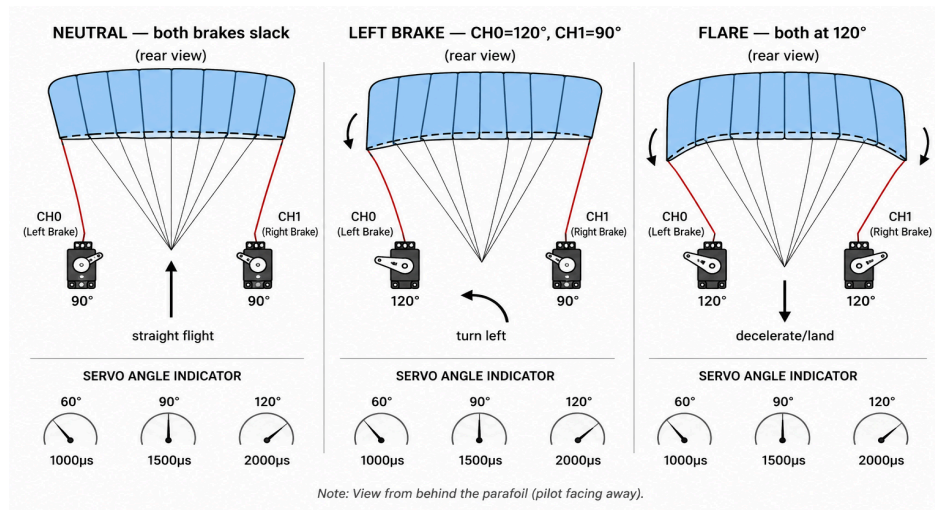


Fig 5. Servo Control Logic

6.1 PCA9685 Channel Map

Channel	Servo	Type	Function
CH0	Left brake	EMAX ES3004	Asymmetric left brake deflection
CH1	Right brake	EMAX ES3004	Asymmetric right brake deflection
CH2	Drogue release	EMAX ES3004	One-shot release at deployment
CH3	Gimbal roll	28BYJ-48	Camera gimbal roll stabilisation
CH4	Gimbal pitch	MG90	Camera gimbal pitch stabilisation

Table 10. PCA9685 Channel Map

6.2 PWM Timing Parameters

EMAX ES3004 (50 Hz):

Position	Angle	Pulse Width
Full brake released (minimum)	60°	1000 μ s
Neutral (no brake applied)	90°	1500 μ s
Full brake applied (maximum)	120°	2000 μ s

Table 11. EMAX ES3004 Parameters

SG90 (50 Hz):

Position	Angle	Pulse Width
Hard limit low (full tilt)	0°	500 μ s
Centre / camera level	90°	1500 μ s
Hard limit high	180°	2400 μ s

Table 12. SG90 Parameters

6.3 Brake Deflection to Servo Angle Mapping

```
if delta_a > 0: # LEFT turn
    CH0_angle = 90 + delta_a    # left brake increases
    CH1_angle = 90             # right brake neutral
if delta_a < 0: # RIGHT turn
    CH0_angle = 90             # left brake neutral
    CH1_angle = 90 + |delta_a| # right brake increases
# Symmetric brake:
    CH0_angle = CH1_angle = 90 + delta_s
# All angles clamped to [60°, 120°] before writing to PCA9685.
```

6.4 Drogue Release Sequence

```
State == APOGEE + deploy trigger:
1. CH2 → 60° (confirm locked position) -- 100 ms hold
2. CH2 → 120° (release)                -- one-shot
```

3. CH2 locked at 120° for remainder of flight (software latch)

7. FAULT DETECTION AND RECOVERY

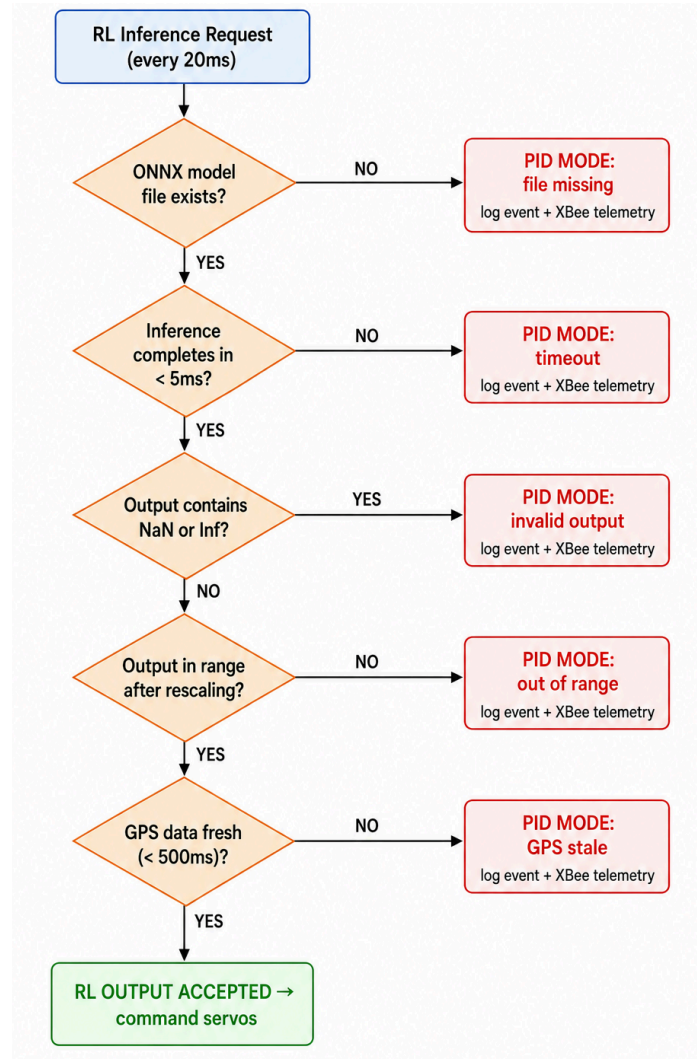


Fig 6. Fault Detection

7.1 RL Model Fault Triggers (Automatic PID Fallback)

Fault Condition	Detection	Action
ONNX model file missing	os.path.exists() at startup	Immediate PID mode
ONNX load exception	try/except on InferenceSession()	Immediate PID mode
Inference timeout > 5 ms	Watchdog timer per step	Switch to PID, log event
NaN or Inf in RL output	np.isfinite() check on output	Switch to PID, log event
Output out of range after rescale	Range check $ \Delta a > 35^\circ$	Switch to PID, log event
GPS data stale > 500 ms	Timestamp check on GPS buffer	Switch to PID, log event

Table 12. Fault Condition Fallback

PID fallback is silent and automatic — no operator intervention required. Every fallback event is logged with timestamp and reason code via XBee telemetry.

7.2 Sensor Fault Handling

Sensor	Fault	Behaviour
BNO085	SPI read exception	Use last known attitude; buzzer warning pattern
BMP388	SPI read exception	EKF propagates from last valid (no measurement update)
GPS	NMEA parse fail	Wind estimator freezes at last estimate; obs[15] degrades
INA219	I2C read fail	Log warning; no control impact
PCA9685	I2C write fail	Emergency buzzer; attempt reconnect next cycle

Table 13. Sensor Fault Handling

7.3 Buzzer Diagnostic Patterns

Pattern	Meaning
2 short beeps	System arm confirmed (ONPAD state)
3 long beeps	Recovery beacon (post-landing)
1 long + 2 short	GPS dropout in flight
Rapid continuous beeps	RL inference fault — PID engaged

Table 14. Buzzer Diagnostic Pattern

7.4 Competition Safety Interlocks

- XBee auto-shutoff: Transmitter silenced exactly 90 minutes after launch detection. Cannot be re-enabled in software without full restart.
- Target coordinate lock: target_latitude and target_longitude can only be written before ONPAD state. After arm, values are frozen in memory for the flight.
- Drogue release lock: CH2 cannot return to 60° after the release event is triggered. One-way software latch is enforced in state machine.